

## 一、1. 区别：

### 可变性：

列表是可变的 (mutable)，这意味着可以在创建列表后修改、添加或删除列表中的元素。例如，可以改变列表中某个元素的值。

元组是不可变的 (immutable)，一旦创建，就不能修改元组中的元素。如果尝试修改元组中的元素，会引发 `TypeError`。

### 语法：

列表使用方括号 `[]` 来定义，例如 `my_list = [1, 2, 3]`。

元组使用圆括号 `()` 来定义，例如 `my_tuple = (1, 2, 3)`。不过，在定义只有一个元素的元组时，需要在元素后面加一个逗号，例如 `my_single_tuple=(1,)`，否则 Python 会将其视为普通的括号表达式。

### 性能：

由于元组是不可变的，在某些情况下，元组的操作可能比列表更快。例如，在作为字典的键时，只能使用不可变类型，元组可以作为字典的键，而列表不行。

### 示例代码：

#### 创建列表：

```
my_list = [1, 2, 3]
my_list.append(4) # 在列表末尾添加元素 4
print(my_list)
```

#### – 创建元组：

```
my_tuple = (1, 2, 3)
# 以下代码会报错，因为元组不可变
# my_tuple[0] = 4
print(my_tuple)
```

### 评分标准：

#### – 区别解释（6分）：

- 可变性解释正确（3 分）
- 语法解释正确（2 分）
- 性能相关解释合理（1 分）
- 示例代码（4 分）：
- 创建列表代码正确（2 分）

## 2. 参数传递方式：

值传递（pass - by - value）：

在 Python 中，基本数据类型（如整数、浮点数、字符串等）是按值传递的。这意味着当把一个基本数据类型的变量作为参数传递给函数时，函数会得到这个变量的一个副本。函数内部对参数的修改不会影响到函数外部的原始变量。

引用传递（pass - by - reference）：

对于可变数据类型（如列表、字典等），Python 是按引用传递的。这意味着当把一个可变数据类型的变量作为参数传递给函数时，函数得到的是这个变量的引用（内存地址）。函数内部对参数的修改会影响到函数外部的原始变量。

示例：

值传递示例：

```
def change_num(num):
```

```
    num = 10
```

```
a = 5
```

```
change_num(a)
```

```
print(a) # 输出 5，因为函数内的修改不影响外部变量
```

引用传递示例：

```
def change_list(lst):
```

```
    lst.append(4)
```

```
my_list = [1, 2, 3]
```

```
change_list(my_list)
```

`print(my_list)` # 输出[1, 2, 3, 4], 因为函数内的修改影响了外部变量

评分标准:

- 参数传递方式解释 (6 分):
- 值传递解释正确 (3 分)
- 引用传递解释正确 (3 分)
- 示例代码 (4 分):
- 值传递示例代码正确 (2 分)
- 引用传递示例代码正确 (2 分)

3. 概念: 装饰器是一种特殊的函数, 它可以在不修改被装饰函数源代码的情况下, 对函数的功能进行扩展或修改。装饰器本质上是一个返回函数的高阶函数。(1 分)

作用:

代码复用: 可以将一些通用的功能, 如日志记录、性能测试、权限验证等, 封装在装饰器中, 然后应用到多个不同的函数上, 避免了在每个函数中重复编写相同的代码。(1 分)

动态修改函数行为: 能够在运行时根据不同的需求改变函数的执行方式, 比如在函数执行前或执行后添加额外的操作。(1 分)

实现原理: 装饰器函数接受一个函数作为参数, 并返回一个新的函数。在返回的新函数内部, 通常会先执行一些额外的操作, 然后再调用被装饰的函数, 并可以对被装饰函数的返回值进行处理后返回。(2 分)

#### 4. 优点:

提高程序的执行效率,能在同一时间内处理多个任务,充分利用多核 CPU 资源,例如在网络爬虫中,可以同时发送多个网络请求来加快数据抓取速度。(1 分)

增强程序的响应性,在处理一些耗时操作时,不会导致整个程序的阻塞,比如在图形界面程序中,一个线程处理后台数据更新,另一个线程负责界面交互,用户界面不会因为数据更新而卡顿。(1 分)

#### 局限性:

由于 GIL (全局解释器锁) 的存在,在 CPython 解释器中,多线程在多核 CPU 上不能真正并行执行 CPU 密集型任务,同一时刻只有一个线程在执行 Python 字节码,所以对于 CPU 密集型任务,多线程可能无法带来显著的性能提升。例如计算大量的数学运算,多线程可能比单线程快不了多少。(2 分)

线程间的同步和通信较为复杂,如果处理不当容易引发死锁、资源竞争等问题,增加程序的复杂性和调试难度。(1 分)

二、1. 答案: 假设 add 函数是对传入的参数进行求和操作,在第一处调用 `calculate(4, 6, 3, 7)` 时,传入的参数分别是  $x = 4$ ,  $y = 6$ ,  $z = 3$ ,  $w = 7$ , 将这些数相加得到结果。在第二处调用 `calculate(4, 6, w = 9)` 时,  $x = 4$ ,  $y = 6$ ,  $z$  使用默认值 2,  $w = 9$ , 再将这些数相加得到结果。

所以最终答案为:

(1) 20 ----- (2 分)

(2) 21 ----- (3 分)

2. 答案: (1) `[3, 4, 5, 6, 7, 8]`

(2) `[6, 7, 8]`

(3) [3, 4, 5, 6, 7]

(4) [5, 6]

(5) True

每空一分

3. **【答案】** Hello, World!

**评分标准：**看输出字段是否正确（5分）

4. **【答案】**

(1) [('Alice', 85), ('Bob', 92), ('Charlie', 78), ('David', 90)]

(2) ['Alice', 'Bob', 'Charlie', 'David']

(3) [78, 85, 90, 92]

(4) ['Alice', 'Bob', 'Charlie', 'David']

(5) [85, 92, 78, 90]

**评分标准：**1. 注意输出格式是否为列表，标点使用是否正确、完整；

2. 注意（3）是否排序

三、1. 答案

```
correct_encryption_key = "secretkey123"
```

```
remaining_attempts = 3
```

```
while remaining_attempts > 0:
```

```
    encryption_key = input("请输入加密密钥: ")
```

```
decryption_key = input("请输入解密密钥： ")

if encryption_key == correct_encryption_key and decryption_key ==
correct_encryption_key:

    print("密钥验证成功，可进行文件操作！ ")

    break

else:

    remaining_attempts -= 1

    if remaining_attempts > 0:

        print(f"请注意剩余尝试次数为 {remaining_attempts} 次")

    else:

        print("3 次密钥输入均有误，无法进行文件操作，请重新获取密钥！

")
```

#### 评分标准

1. 正确定义正确密钥及初始化剩余次数（2 分）：准确地定义了正确的加密密钥和解密密钥（这里二者相同），同时合理初始化了剩余尝试次数的变量。
2. 正确获取用户输入（2 分）：通过 input 函数分别获取用户输入的加密密钥和解密密钥，代码简洁明了，不存在获取输入方面的语法错误或逻辑混乱情况。
3. 准确实现验证逻辑（4 分）：正确比对用户输入的加密密钥和解密密钥与预设的正确密钥是否一致（2 分），能准确判断匹配情况进入相应逻辑分支。根据剩余尝试次数合理控制循环和输出不同提示的逻辑（2 分），确保整个验证过程的逻辑完整性，根据不同验证结果准确执行对应操作。
4. 正确输出提示语句（2 分）：针对密钥验证成功、还有剩余尝试次数验证失败、超过尝试次数验证失败这几种场景，能准确输出对应的提示内容，不存在提示信息格式错误或与要求不符的情况。

2. 答案:

```
p = 1 ---1'
print(f'第 10 天还剩下 {p} 个桃子') ---2'
for i in range(9,0,-1):
    p = (p + 1) * 2
    print(f'第 {i} 天还剩下 {p} 个桃子') ---5'
print(f'第一天一共摘了 {p} 个桃子') ---7'
```

```
# 第 10 天还剩下 1 个桃子
# 第 9 天还剩下 4 个桃子
# 第 8 天还剩下 10 个桃子
# 第 7 天还剩下 22 个桃子
# 第 6 天还剩下 46 个桃子
# 第 5 天还剩下 94 个桃子
# 第 4 天还剩下 190 个桃子
# 第 3 天还剩下 382 个桃子
# 第 2 天还剩下 766 个桃子
# 第 1 天还剩下 1534 个桃子
```

```
第一天一共摘了 1534 个桃子 ---10'
```

3. while True:

```
s = input("请输入一个字符串（输入'quit'退出）： ")
if s == "quit":
    break
s = s.lower() # 转换为小写，不区分大小写判断回文
s = s.replace(" ", "") # 去掉空格
if s == s[::-1]:
```

```
print(f'{s}是回文串。")
else:
    print(f'{s}不是回文串。")
```

评分标准如下：从功能方面来看，占 6 分，其中输入处理正确（包括能按要求获取用户输入并实现 'quit' 退出机制，以及正确处理大小写输入）可得 2 分；准确地去除空格并判断字符串是否为回文可拿 3 分，若还能正确处理空字符串则再加 1 分；能正确输出判断结果得 1 分。代码风格方面占 3 分，代码缩进正确且空格使用合理可得 1 分；有恰当且清晰准确的注释可得 1 分；变量命名符合规范且有意义可得 1 分。在效率方面占 1 分，若回文判断算法的时间复杂度为  $O(n)$ （ $n$  为字符串长度），则可获得这 1 分。

#### 4. 答案

```
import os

try:
    file_path = 'H:\\quote.txt'
    try:
        with open(file_path, 'w') as f:
            lines = ["Errors should never pass silently.", "Unless explicitly silenced.", "In the face of ambiguity, refuse the temptation to guess."]
            for line in lines:
                f.write(line + '\n')
        with open(file_path, 'r+') as f:
            content = f.read().upper()
            f.seek(0)
            f.write(content)
    except FileNotFoundError:
        file_path = 'quote.txt'
        with open(file_path, 'w') as f:
```



```
lines = ["Errors should never pass silently.", "Unless explicitly  
silenced.", "In the face of ambiguity, refuse the temptation to guess."]
```

```
for line in lines:
```

```
    f.write(line + '\n')
```

```
with open(file_path, 'r+') as f:
```

```
    content = f.read().upper()
```

```
    f.seek(0)
```

```
    f.write(content)
```

```
except PermissionError:
```

```
    print('没有足够的权限创建或操作文件')
```

评分标准：

满分十分，从功能、异常处理、代码结构三方面考量。功能上，能正确写入三行语句到文件并成功读取转换大写后重写得 4 分，写入或转换重写有误酌情扣 1 - 2 分；异常处理方面，FileNotFoundError 处理正确得 2 分，错误则扣 1 - 2 分，PermissionError 捕获且不导致程序崩溃有一定逻辑得 2 分，无有效处理则扣 1 - 2 分；代码结构上，存在重复代码未优化扣 1 - 2 分，代码风格规范（缩进、命名、书写风格）不佳扣 1 - 2 分。

#### 四、1. # 参考答案

```
def process_employees(data):
```

```
    total_promoted = 0
```

```
    total_remained = 0
```

```
    employees_file = open("employers.txt", "w")
```

```
    for i, (age, score) in enumerate(data, start=1):
```

```
        if age >= 35:
```

```
            if score >= 80:
```

```
                result = "晋升"
```

```
                total_promoted += 1
```

```

        else:
            result = "原岗"
            total_remained += 1
    else:
        if score >= 90:
            result = "晋升"
            total_promoted += 1
        else:
            result = "原岗"
            total_remained += 1

    employees_file.write(f'{i}, {age}, {score}, {result}\n')

employees_file.close()

print(f"总晋升人数: {total_promoted}, 原岗人数: {total_remained}")

n = int(input())
data = []
for i in range(n):
    age, score = map(int, input().split())
    data.append((age, score))
process_employees(data)
print("文件依次存储工号，年龄，考核分，考核结果")
with open("employers.txt", "r") as file:
    print(file.read())

```

- 评分标准：
1. 正确实现输入操作相关代码：2 分
  2. 正确创建文件并写入相关信息：3 分
  3. 正确实现文件读的操作：2 分
  4. 输出晋升人数和原岗人数正确：3 分

5. 输出员工信息格式正确：5 分

2. # 参考答案

```
def process_employees(data):  
    total_promoted = 0  
    total_remained = 0  
    employees_file = open("employers.txt", "w")  
  
    for i, (age, score) in enumerate(data, start=1):  
        if age >= 35:  
            if score >= 80:  
                result = "晋升"  
                total_promoted += 1  
            else:  
                result = "原岗"  
                total_remained += 1  
        else:  
            if score >= 90:  
                result = "晋升"  
                total_promoted += 1  
            else:  
                result = "原岗"  
                total_remained += 1  
  
        employees_file.write(f'{i}, {age}, {score}, {result}\n')  
  
    employees_file.close()
```

```
print(f'总晋升人数: {total_promoted}, 原岗人数: {total_remained}')

n = int(input())
data = []
for i in range(n):
    age, score = map(int, input().split())
    data.append((age, score))
process_employees(data)
print("文件依次存储工号, 年龄, 考核分, 考核结果")
with open("employers.txt", "r") as file:
    print(file.read())
```

评分标准: 1. 正确实现输入操作相关代码: 2 分

- 2. 正确创建文件并写入相关信息: 3 分
- 3. 正确实现文件读的操作: 2 分
- 4. 输出晋升人数和原岗人数正确: 3 分
- 5. 输出员工信息格式正确: 5 分